

Тестовое задание: LLM Harness Engineering

Архитектура исполнительного контура (Harness): метапромпты, субагенты и динамические Skills

Трудозатраты: 3–5 часов

Стек: Python

AI Core:

Sub-Agent Orchestration & Skills Loading

Контекст: Raw LLM ограничена статическим контекстом. Чтобы решать сложные индустриальные задачи, вокруг модели строится **Harness** — среда исполнения, управляющая декомпозицией, вызовом субагентов и динамическим подключением предметных навыков (Skills). Ваша задача — разработать рабочий прототип Harness для одного выбранного домена.

ОБЯЗАТЕЛЬНОЕ ЯДРО HARNESS (MUST-HAVE)

1 Системный метапромпт & Диспетчер задач

Центральный оркстратор (Root Agent), который принимает комплексный запрос пользователя, определяет необходимый план действий и распределяет подзадачи между субагентами.

2 Оркестрация субагентов (Sub-Agents Calling)

Иерархическое исполнение: диспетчер вызывает минимум 5 специализированных субагента (например: Генератор/Исполнитель и Ревьюер/Валидатор или два узких эксперта подэтапов).

3 Механизм динамических навыков (Skills Injection)

Возможность загружать внешние Skills (модульные описания методик, контекстные правила, схемы рассуждений) и передавать их целевому субагенту вместе с задачей (runtime-инъекция).

Опционально: песочница выполнения кода, RAG/векторный поиск, веб-поиск или кастомные тулы.

ПРЕДМЕТНАЯ ОБЛАСТЬ (1 НА ВЫБОР)

Выберите наиболее близкое вам прикладное направление:

1. Software Engineering. Генерация/исправление кода. Субагенты: генератор решения + ревьюер кода / генератор тестов. Скиллы: стандарты SE/безопасности, шаблоны тестирования.

2. Medical / Health Data. Анализ клинического случая. Субагенты: диагност + аудитор противопоказаний. Скиллы: протоколы дозирования, правила назначения антибиотиков.

3. Finance & Compliance. Анализ фин. транзакций. Субагенты: аналитик аномалий + аудитор регуляторных рисков. Скиллы: правила проверки кассовых разрывов, правила вычета налогов.

4. Свой домен (Math / Science / System Design). Любое сложное направление при сохранении требований к иерархии субагентов и скиллам.

ТРЕБОВАНИЯ К СДАЧЕ (DELIVERABLES)

- GitHub Репозиторий:** чистый код на Python (FastAPI / CLI / LangGraph / Pydantic / LiteLLM / чистый API клиент).
- Каталог навыков (/skills):** минимум 2 оформленных скилла (в виде JSON/Markdown или классов), динамически передаваемых субагентам.
- Исполняемый запуск (main.py):** воспроизводимый скрипт, прогоняющий 1–2 тестовых кейса в консоли с логами делегирования субагентам.
- README.md (1 стр.):**
 - Архитектурная схема Harness (диспетчер → скиллы → субагенты).
 - Лог работы: наглядная демонстрация вызова субагента с подключенным скиллом.

КРИТЕРИИ ОЦЕНКИ (SCORING)

Критерий	Вес	Фокус проверки
Архитектура Harness	35%	Качество метапромпта, реальное разделение ролей между диспетчером и субагентами.
Механизм Skills	30%	Модульность навыков, чистота их динамической передачи и влияния на результат субагента.
Адекватность домену	20%	Логичность сценария и применимость разработанной обвязки к выбранной индустрии.
Качество кода & Setup	15%	Чистота Python-архитектуры, модульность, запуск «из коробки» по README.